Making a Password Blender

Password Mutations

SUGRCAIN

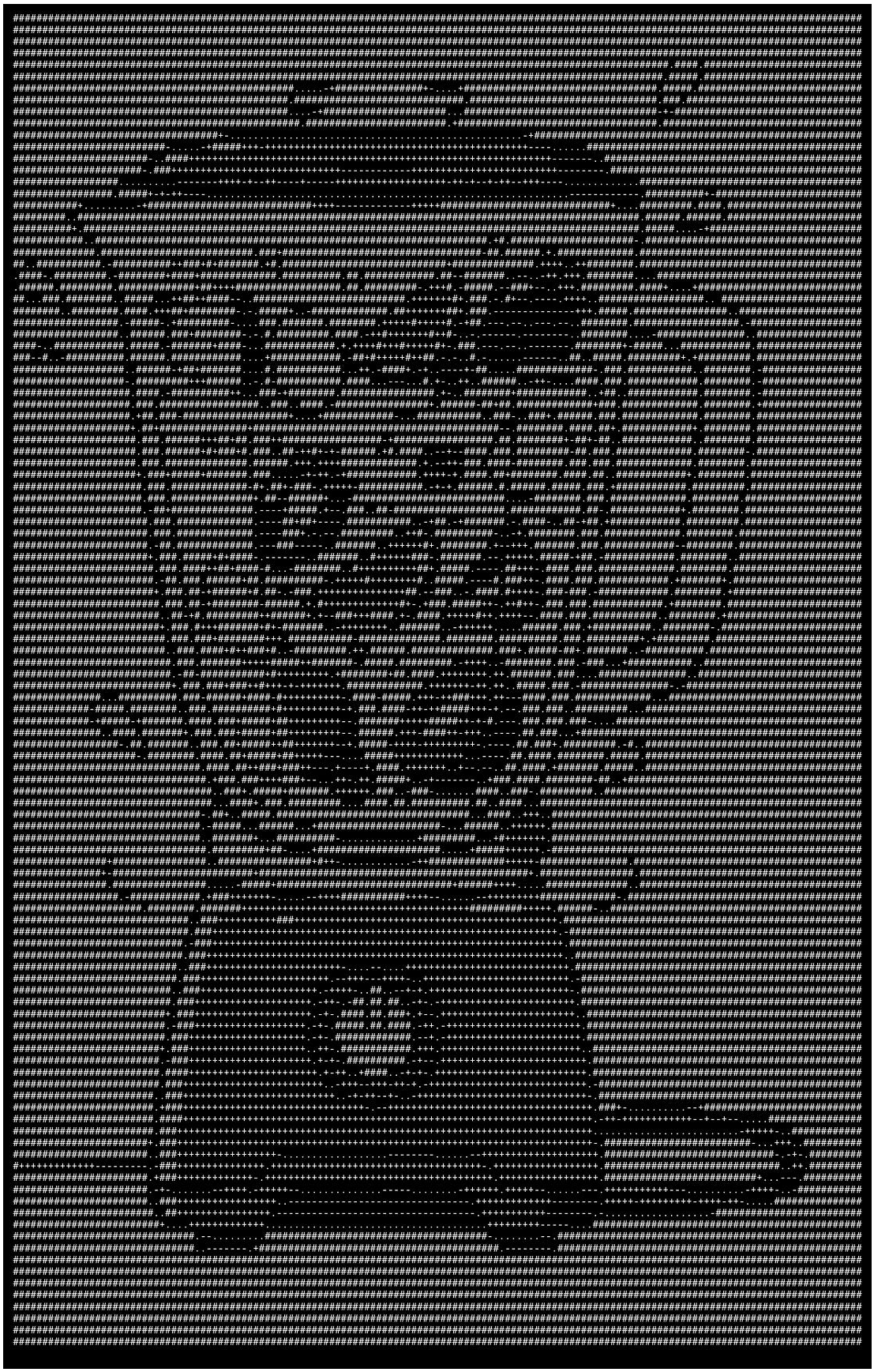
JUL 13, 2026

In this blog, we will delve into the crucial aspects of building potent wordlists and creating mutated passwords for effective penetration testing. We will explore the significance of a well-constructed wordlist in identifying vulnerabilities within a system, emphasizing the importance of diversity and relevance. Additionally, we will discuss the concept of mutated passwords. How slight modifications to common passwords can counter traditional security measures.

Making a WordList Toolkit

Before we begin mutating, we should have exhausted wordlists with the best chance of success before delving into complicated wordlist mutation. We could try default password lists, data breach leaks, or good ole common password lists. **Mutation might not be necessary**.

Keep it simple, Occam’s razor.



Default Password Lists

There are occasions when a system administrator may overlook aspects of the configuration process, resulting in misconfigurations that we can exploit and report. One of these misconfigurations is the use of **default passwords, and it's more common than you might think.**

Here are a few downloadable cheat sheets to get you started:

ihebski - CheatSheet

- *A default credential tool aggregated from several sources.*

SecLists - Default Credentials

- *Provides multiple types of lists for engagements (usernames, passwords, URLs, web shells, and more).*

Google Dorking

- *And some Google searching, but that should be a given.*

Data Breaches

Effective wordlists can also stem from data breaches, provided it is permissible and explicitly outlined in the Scope of Engagement (SOE). An illustrative instance of this is the *rockyou.txt* wordlist, which originated from a significant data breach suffered by RockYou. The RockYou data breach in 2009 serves as just one example; however, data breaches have now become commonplace. Verizon's annual Data Breach Index Report reveals that as much as **80% of successful data breaches stem from compromised login credentials**. I won't be providing direct links to any wordlists. However, if you're curious, you

can check haveibeenpwned.com to see if your password appears in a data breach wordlist, somewhere in their database.

Mutated Passwords

Password mutations refer to the modification of passwords based on predefined rules that mimic those seen by the target system. By testing how well a system withstands mutated password attempts, we can identify vulnerabilities and weaknesses in the authentication process. This, in turn, helps organizations enhance their overall security posture by addressing potential points of exploitation and reinforcing robust password policies.

For this example, we will use Microsoft's definition of what a strong password looks like. According to Microsoft, a strong password **contains 12 characters and a combination of symbols, numbers, and letters.**

> Note that WPengine states that most passwords are not longer than ten characters.

Create the Rules

We will use Hashcat and its “Rule-based Attack” documentation to create a `.rule` file, which allows us to mutate an existing password list to match the authentication requirements of our target.

It could look something like this:

```
$ mousepad custom.rule&
```

```
:
p2
fp2
dC$!
```

Or, we could use Hashcat’s existing files:

```
$ ls /usr/share/hashcat/rules/
```

```
Incisive-leetspeak.rule                  generated.rule
InsidePro-HashManager.rule               generated2.rule
InsidePro-PasswordsPro.rule              hybrid
T0XIC-insert_00-99_1950-2050_toprules_0_F.rule  leetspeak.rule
T0XIC-insert_space_and_special_0_F.rule       oscommerce.rule
T0XIC-insert_top_100_passwords_1_G.rule       rockyou-30000.rule
T0XIC.rule                               specific.rule
T0XIC_3_rule.rule                        toggles1.rule
T0XIC_insert_HTML_entities_0_Z.rule          toggles2.rule
T0XICv2.rule                             toggles3.rule
best64.rule                              toggles4.rule
combinator.rule                          toggles5.rule
```

```
d3ad0ne.rule                    unix-ninja-leetspeak.rule
dive.rule
```

Or, some good ole Google Dorking

Google Dorking

Prep Your List

You can have an existing wordlist or create a custom wordlist that you're ready to mutate. For this example, I'm going to use a wordlist that contains just one word so we can observe the effects of the mutation.

```
$ echo Password > password.list
```

Mutate !

Hashcat offers a range of flags to provide additional specifications for your new wordlist. However, in this example, I will specifically ensure that each password in the list consists of exactly 12 characters.

```
$ hashcat --force password.list -r custom.rule --stdout | sort -u | grep -vwE
'\w{1,11}' > mut_password.list
```

And Ta-da !

```
$ cat mut_password.list

PasswordPasswordPassword
PassworddrowssaPPassworddrowssaPPassworddrowssaP
pASSWORDPASSWORD!
```